Restaurant OS

Master Product Requirements & Developer Handoff

Version 0.1 | 23 August 2026

Replace the merchant's software, not the merchant's cafe.

A multi-tenant restaurant operating and revenue platform, initially optimized for Indian cafes and extensible to restaurants, bakeries, QSRs, cloud kitchens and multi-outlet brands.

This document is the product and engineering source of truth for initial implementation. Unresolved decisions are explicitly marked and must become issues/ADRs rather than being invented in code.

1. Vision

Restaurant OS is the operating system and revenue engine for Indian cafes and restaurants.

The long-term product replaces legacy POS/restaurant software while adding capabilities those systems traditionally treat as separate products: branded digital storefronts, direct ordering, customer CRM, loyalty, gamification, referrals, memberships, gift cards, campaigns and revenue intelligence.

2. Core promise

The merchant should not have to rebuild their business around the software. The software should adapt to the merchant's business while replacing the merchant's fragmented software stack.

3. Product pillars

1. Operate the restaurant: POS, billing, payments, tables, KOT/KDS, inventory, recipes, procurement and staff.
2. Sell more: direct ordering, QR, recommendations, bundles, promotions and upsells.
3. Retain customers: CRM, loyalty, games, streaks, memberships and reordering.
4. Grow intelligently: segmentation, campaigns, referrals, analytics, reconciliation and rule-based automation.
5. Integrate: aggregators, payment providers, accounting, messaging and hardware.

4. North-star merchant outcome

Incremental gross profit influenced by the platform.

Secondary outcomes:

- Repeat purchase rate.
- Average order value.
- Direct-order share.
- Customer lifetime value.
- Gross margin.
- Wastage.
- Merchant retention.

5. Product philosophy

- Lowest practical merchant friction.
- Guest-first customer ordering.
- Merchant-controlled economics.
- Offline-safe operations.
- Auditability of money and rewards.
- Multi-tenant from day one.
- Integration-friendly core architecture.
- No mandatory AI dependency.
- Rules and data before expensive AI.
- Feature breadth should follow proven merchant value.

6. Strategic positioning

Do not position the company as merely a cheaper POS or a website builder.

Positioning:

"A complete restaurant operating and customer-growth system that replaces your old software and helps you generate more repeat revenue."

Migration message:

"Replace your software, not your cafe."

Version: 0.1

Status: Developer handoff draft

Audience: Product, engineering, design, QA, operations

1. Product definition

Restaurant OS is a multi-tenant SaaS platform for restaurants. Initial GTM is independent Indian cafes and restaurants with roughly 30-50 orders/day, but the domain model must support larger merchants and multiple outlets.

Each paid outlet receives an operating environment containing POS and operational modules plus a customer-facing branded web storefront and QR ordering experience.

Example:

```
mukulscafe.ourbrand.com
```

The system must be capable of eventually replacing legacy POS software rather than merely sitting beside it.

2. Primary users

2.1 Owner

Needs financial visibility, configuration, high-level performance, permissions, customer growth and exception handling.

2.2 Manager

Runs daily operations, staff, inventory, orders, campaigns and reports.

2.3 Cashier

Creates and settles orders, handles payments and receipts, with restricted access to sensitive settings.

2.4 Kitchen staff

Receives KOT/KDS work and changes preparation state.

2.5 Captain/front-of-house

Creates/updates table orders, sees service state and customer requests where applicable.

2.6 Accountant

Read/report access to invoices, tax records, settlements and exports.

2.7 Customer

Browses, orders, pays, earns/redeems rewards, plays optional games and receives offers.

2.8 Platform admin

Manages tenants, subscriptions, support, feature flags, integration health, fraud and system operations.

3. Tenant model

Top-level tenant = merchant brand/business.

Each tenant contains one or more outlets.

Subscription billing is per outlet.

Tenant data must never cross tenant boundaries. Outlet-level operational data is isolated while brand-level rollups are available to authorized brand users.

4. Pricing requirements

Launch target: INR 499/month/outlet.

Expected future tiers:

- Starter: 499
- Growth: approximately 999
- Pro: approximately 1,999

These numbers are commercial targets, not hardcoded product constants.

Subscription-first. Hardware, payment processing, WhatsApp/SMS and premium integrations may be optional revenue streams.

5. Core modules

P0

- Merchant onboarding
- Multi-tenancy
- Menu/catalog
- POS
- Order lifecycle
- Billing/invoicing
- Payments recording and configurable gateway support
- Offline operation for critical POS workflows
- KOT
- Basic KDS
- Tables/floor basics
- Inventory basics
- Recipe basics
- Staff and permissions
- Thermal printer support architecture
- Customer web storefront
- QR ordering
- Guest checkout
- Customer profiles keyed primarily by mobile number

- Loyalty engine and ledger
- Coupons/promotions
- Merchant dashboard
- Subscription billing
- Audit log

P1

- Swiggy/Zomato integrations subject to official API/partner capability
- Deep inventory
- Procurement
- Supplier management
- Reconciliation
- Referral engine
- Memberships
- Gift cards
- Gamification engine
- Campaign automation
- Advanced analytics
- Reviews/feedback
- Multi-outlet operations
- Advanced KDS
- Accounting exports

P2

- Predictive churn
- Demand forecasting
- Dynamic merchandising
- AI assistant
- Franchise controls
- Hardware marketplace
- Consumer discovery/network

6. Merchant onboarding

Target: a cafe can reach a testable live state within one onboarding session.

Flow:

6. Create account.
7. Choose plan.
8. Create tenant/outlet.
9. Enter business details.
10. Configure taxes and invoice information.
11. Configure payment methods.
12. Import or create menu.
13. Configure tables and kitchen stations.
14. Connect printers/devices.
15. Run test order.
16. Print test KOT and receipt.

17. Publish customer site and QR codes.
18. Go live.

Migration should support a parallel period where the merchant can validate Restaurant OS against their old workflow before cutover.

7. Merchant website

Each outlet gets a branded storefront. Default model is structured templates with approximately 80% structured configuration and 20% customization.

Required capabilities:

- branding
- menu/category display
- search
- product details
- modifier selection
- recommendations
- cart
- checkout
- payment
- order status
- offers
- rewards visibility
- games entry point
- referrals
- reorder
- gift cards/membership where enabled

Custom domains are a future capability but the routing architecture must support them.

8. Customer ordering

Guest-first:

19. Scan/open cafe site.
20. Browse without login.
21. Add items.
22. See cross-sells and bundles.
23. Checkout.
24. Choose available payment method.
25. Optionally provide mobile number for customer identity/rewards.
26. Receive order confirmation.

Mobile number is the primary customer identifier within a merchant relationship. OTP should not be mandatory for ordinary ordering because messaging costs should be minimized.

9. Order engine

All order sources use one canonical order model:

- POS
- QR
- website
- aggregator integrations
- future phone/manual sources

Canonical status flow:

DRAFT -> PLACED -> ACCEPTED -> PREPARING -> READY -> SERVED/PICKED_UP -> COMPLETED

Exceptional states:

CANCELLED, FAILED, REFUNDED, PARTIALLY_REFUNDED.

Every state transition is auditable and event-driven.

10. POS requirements

Support:

- dine-in
- takeaway
- delivery
- table order
- counter order
- modifiers
- add-ons
- combos
- split bill
- merge bill
- hold/resume
- price override permissions
- discounts
- taxes/service charges/tips where configured
- refunds and partial refunds
- receipt reprint
- duplicate receipt rules
- customer assignment
- payment methods
- cash drawer/opening/closing cash
- end-of-day settlement
- order notes

11. Offline requirements

Critical POS workflows must remain operational during internet loss.

At minimum while offline:

- create order
- modify order

- send KOT/KDS work where local network topology permits
- print supported receipts/KOT
- record cash/card/UPI status according to configured mode
- close order
- queue synchronization

Every local transaction must carry a globally unique identifier and deterministic idempotency key.

On reconnection:

- upload pending events
- reconcile inventory
- reconcile payment state
- avoid duplicate orders
- record conflicts
- preserve original timestamps and operator/device context

12. Kitchen

Capabilities:

- KOT print
- KDS screen
- station routing
- item readiness
- prep timer
- priority
- sold-out/unavailable items
- delayed order indicators
- completion state

13. Tables

Capabilities:

- floor plan
- table status
- table QR
- occupied/available/reserved/cleaning
- move order
- merge tables
- split bill by guest/item
- multiple orders per table

14. Inventory

Core:

- ingredients
- units and conversions
- stock on hand

- stock adjustments
- wastage
- damage
- transfers
- reorder thresholds
- stock counts

Recipe consumption:

Product sale decrements component ingredients using configured recipes.

15. Procurement

P1:

- suppliers
- purchase orders
- goods received
- purchase invoices
- supplier pricing history
- purchase returns
- accounts payable export

16. Billing and finance

Capabilities:

- invoice generation
- invoice numbering
- business details
- tax configuration
- credit/debit notes where required
- payment status
- refunds
- settlement records
- sales summaries
- payment channel summaries
- aggregator reconciliation

Exact GST compliance rules must be validated during implementation against current Indian requirements and qualified accounting advice.

17. Customer CRM

Customer profile may contain:

- mobile number
- name
- optional email
- visit count
- order count

- total spend
- average order value
- last visit
- favorite categories/products
- loyalty balance
- coupons
- membership
- referrals
- feedback
- consent/preferences

Segments:

- new
- active
- loyal
- VIP
- at-risk
- dormant
- high-value
- category preference
- frequency bands

18. Loyalty engine

Merchant configurable rules:

- spend-based points
- first order
- repeat visits
- referral
- birthday
- time/day multipliers
- product-specific rewards

Merchant controls:

- earn rate
- redemption value
- expiry
- transferability
- redemption restrictions

System must warn about unusually generous economics and show estimated effective discount where possible.

All points changes are ledger entries, never silent balance mutations.

19. Promotion engine

Unified engine for:

- percentage discounts
- fixed discounts
- minimum order value
- item offers
- bundles
- BOGO
- time-based offers
- customer segment offers
- membership offers
- loyalty bonuses
- game rewards

Rules must be deterministic, auditable and precedence-controlled to prevent stacking bugs.

20. Recommendations

Phase 1:

- merchant-defined associations
- bundles

Phase 2:

- co-purchase analytics
- customer preference recommendations

Recommendations should be explainable enough for merchant operators to understand why an item is being promoted.

21. Gamification

Build a generic game engine rather than hard-code one game.

Game configuration:

- game type
- attempts/day
- cooldown
- eligibility
- skill/chance model
- reward type
- win configuration
- minimum order requirements
- expiry
- merchant budget

Separate game currency from monetary loyalty points. Game coins may be used for participation/engagement; loyalty points represent merchant-funded value.

Potential games:

- scratch card

- spin
- memory
- timing
- quiz
- cafe rush
- lucky table
- daily challenge

Games must include fraud/rate-limit protections and merchant spending controls.

22. Referral engine

Flow:

Customer receives referral link/code -> friend lands on cafe site -> friend orders -> attribution recorded -> reward triggered according to merchant rule.

Need:

- unique referral identifiers
- attribution window
- fraud prevention
- reward ledger entry
- reporting

23. Memberships

Future/P1:

- recurring membership
- benefits
- included items/credits
- discounts
- points multipliers
- renewal/expiry

24. Gift cards

Future/P1:

- issue
- send/share
- redeem
- partial redemption
- expiry
- balance
- refund rules

25. Campaign engine

Manual and automated campaigns.

Manual:

- segment
- offer
- channel
- schedule
- cap

Automated triggers:

- first order
- second-order incentive
- inactivity
- birthday
- high value
- frequent buyer
- product-specific event
- new product
- inventory event

Messaging should be provider-agnostic. Paid messaging channels should be configurable by plan.

26. Merchant dashboard

Home must answer:

27. How much did I sell?
28. How did I perform vs baseline?
29. What is going wrong?
30. Where am I missing revenue?
31. What should I do next?

Suggested top-level cards:

- today's sales
- orders
- average order value
- repeat rate
- direct order share
- gross margin estimate
- alerts
- revenue opportunities

27. Revenue opportunities

System should surface deterministic opportunities such as:

- high-value inactive customers
- under-selling profitable products
- low-margin popular products
- high stock/wastage risk
- unused direct ordering channel
- bundle opportunities

Do not require generative AI.

28. Staff

P0:

- users
- roles
- permissions
- shift/login audit
- cashier reconciliation
- device association
- restricted actions

P1:

- attendance
- scheduling
- leave
- incentives
- performance

29. Hardware

Bring-your-own is default. Support a hardware abstraction layer.

Initial target classes:

- thermal printer
- kitchen printer
- cash drawer
- barcode scanner
- Windows terminal
- Android tablet

Hardware purchasing is optional future revenue.

30. Aggregator integrations

Architect around adapters so core order/domain logic does not know vendor-specific details.

Target integrations: Swiggy and Zomato.

Potential capabilities:

- order ingestion
- status updates
- menu sync
- availability
- price sync
- reporting
- reconciliation

Actual availability depends on official APIs/partnerships and must be verified during integration work. Do not design the core product around unofficial access.

31. Accounting integrations

P1:

- exports
- Tally/Zoho-compatible workflows where technically appropriate
- invoice data
- purchase data
- settlement data

32. Security

Required:

- tenant isolation
- role-based access control
- secure authentication
- MFA for owners as an option/strong recommendation
- audit logs
- encryption in transit/at rest as appropriate
- backups
- disaster recovery
- sensitive-action reauthentication
- rate limiting
- fraud controls

33. Data ownership

Merchant-specific customer relationship data is treated as merchant-owned business data. Product terms must define access, export, retention and deletion rights.

Merchant export should cover at least:

- customers
- orders
- products
- loyalty ledger
- invoices/reports

34. Admin platform

Platform admin capabilities:

- merchant management
- subscription management
- plan configuration
- support tooling
- safe support impersonation with audit
- feature flags

- integration health
- billing support
- fraud review
- system health

35. Success criteria

Launch is successful when a representative 30-50 order/day merchant can operate core daily transactions without depending on a legacy POS, and can publish a customer ordering channel with loyalty/CRM.

Product-market validation should measure:

- activation rate
- time to first order
- time to go live
- merchant retention
- weekly active merchants
- direct order volume
- repeat purchase rate
- average order value
- support tickets/merchant
- critical operational failure rate

36. Non-goals for V1

- universal consumer marketplace
- mandatory AI
- full enterprise accounting suite
- proprietary payment rail
- mandatory hardware bundle
- complex reservation marketplace
- franchise ERP
- social network

Roles

- Owner
- Manager
- Cashier
- Captain
- Kitchen
- Accountant
- Marketing
- Platform Admin

Permission principles

32. Least privilege.
33. Money-changing actions require elevated permission.

34. Configuration changes are audited.
35. Owner can grant/revoke permissions except platform-only controls.
36. Outlet scope applies to outlet users.

Example matrix

Capability	Owner	Manager	Cashier	Captain	Kitchen	Accountant	Marketing
Create orders	Yes	Yes	Yes	Yes	No	No	No
Take payment	Yes	Yes	Yes	Optional	No	No	No
Refund	Yes	Configurable	Limited	No	No	Limited	No
Change menu	Yes	Yes	No	No	No	No	No
Change price	Yes	Configurable	No	No	No	No	No
View sales	Yes	Yes	Limited	Limited	Limited	Yes	Campaign view
View profit	Yes	Configurable	No	No	No	Yes	No
Manage inventory	Yes	Yes	No	No	No	Read	No
Manage staff	Yes	Yes	No	No	No	No	No
Manage campaigns	Yes	Yes	No	No	No	No	Yes
Manage loyalty	Yes	Yes	No	No	No	No	Yes
Configure payments	Yes	Limited	No	No	No	No	No
Configure tax	Yes	Limited	No	No	No	Yes	No
View KDS	Yes	Yes	Yes	Yes	Yes	No	No

Final permissions must be implemented as granular capabilities, not hard-coded role checks.

A. Merchant onboarding

Signup -> plan -> tenant -> outlet -> business profile -> menu -> taxes -> payments -> printers/devices -> test order -> QR -> publish.

B. Customer QR order

Scan -> storefront -> menu -> item -> modifier -> recommendation -> cart -> mobile number optional -> payment -> order confirmation -> preparation -> pickup/serve -> reward -> optional game -> return.

C. POS order

Login -> choose order type -> table/customer optional -> add products -> modifiers -> promotion validation -> payment -> invoice -> KOT -> kitchen -> completion -> ledger/events.

D. Offline order

Device detects offline -> local order -> local validation -> local print -> local state -> sync queue -> connection restores -> idempotent upload -> server ack -> local tombstone/mark synced.

E. Refund

Authorized user -> locate order -> choose whole/partial -> reason -> refund payment where supported -> reverse loyalty/points if applicable -> update inventory if relevant -> audit -> customer notification.

F. Loyalty redemption

Customer identity -> view balance -> validate reward -> reserve/redeem -> apply discount -> ledger entry -> completion. Failed transaction must release reservation.

G. Referral

Customer generates code -> friend lands -> attribution cookie/token -> eligibility -> first qualifying order -> reward ledger -> notification.

H. Campaign

Merchant selects segment -> offer -> channel -> schedule -> approval -> send/queue -> delivery status -> redemption attribution -> campaign report.

I. Swiggy/Zomato order

Provider adapter receives event -> verify authenticity -> map provider order -> canonical order -> KOT/KDS -> payment status -> provider status update -> reconciliation event.

J. Migration

Import source data -> mapping -> validation -> exception list -> test environment -> parallel run -> go-live checkpoint -> final cutover -> legacy archival.

Merchant Web / POS

37. Home
38. POS
39. Orders
40. Kitchen
41. Tables
42. Menu
43. Inventory
44. Procurement
45. Customers
46. Marketing
47. Loyalty & Rewards
48. Games
49. Memberships
50. Gift Cards
51. Analytics
52. Reconciliation

53. Staff
54. Settings
55. Integrations
56. Support

Customer Web

57. Home
58. Menu
59. Product detail
60. Cart
61. Checkout
62. Order status
63. Rewards
64. Games
65. Offers
66. Orders
67. Profile/preferences
68. Referral
69. Membership
70. Gift cards
71. Feedback/support

Platform Admin

72. Tenants
73. Outlets
74. Subscriptions
75. Usage
76. Integrations
77. System health
78. Support
79. Fraud
80. Feature flags
81. Audit

Navigation rules

- POS and active order execution must be reachable in one interaction.
- Merchant home is operational summary, not a report catalog.
- Advanced settings should be separated from daily operation.
- Customer storefront should require no account for browsing/order initiation.

POS

Objective

Process restaurant transactions reliably, including when internet connectivity is unavailable.

Functional requirements

- Create/edit/hold orders.
- Multiple order types.
- Modifiers and notes.
- Pricing and promotion evaluation.
- Table assignment.
- Customer attachment.
- Payment capture/recording.
- Invoice/receipt.
- KOT/KDS event.
- Refund/void according to permission.

Acceptance criteria

- 100 consecutive local transactions can be created in an offline test scenario without data loss.
- Reconnect must not duplicate orders.
- Every financial adjustment has an operator and reason.

Menu/catalog

- Categories.
- Products.
- Variants/modifiers.
- Price by outlet.
- Availability.
- Images.
- Recipe mapping.
- External channel mappings.

Customer CRM

- Merchant-scoped customer profile.
- Mobile number primary identity.
- Guest order possible.
- Duplicate-phone resolution workflow.
- Purchase summary.
- Segmentation.

Loyalty

- Rule engine.
- Ledger.
- Balance computation.
- Expiry.
- Redemption restrictions.
- Refund reversal.
- Merchant economics warning.

Games

- Game registry.
- Configuration.

- Attempt policy.
- Reward policy.
- Session tracking.
- Fraud/rate limiting.
- Event logging.

Campaigns

- Audience builder.
- Offer attachment.
- Schedule.
- Approval.
- Delivery adapter.
- Redemption attribution.

Analytics

Required views:

- Sales by day/hour.
- Orders by channel.
- AOV.
- Repeat rate.
- Product sales.
- Gross margin estimate.
- Discount impact.
- Customer cohorts.
- Loyalty usage.
- Campaign performance.
- Aggregator reconciliation.

Support/recovery

- Issue ticket linked to order/customer.
- Refund/replacement/coupon resolution.
- Internal notes.
- Resolution audit.

Commercial

- Launch target is INR 499/month/outlet.
- Subscription is primary revenue stream.
- No mandatory hardware purchase.
- No mandatory payment gateway.
- Direct-order transaction fee target is zero initially.
- Premium messaging/integrations may be tiered.

Customer

- Browsing does not require login.

- Ordering should support guest mode.
- Mobile number is the preferred merchant-scoped customer identity.
- OTP is not mandatory for ordinary ordering.
- Customer data does not automatically become a platform-wide consumer profile.

Loyalty

- Merchant defines earn/redemption rules.
- Every points change is a ledger transaction.
- Refunds may reverse earned points according to configured rules.
- Expiry/transferability are merchant-controlled where supported.

Promotions

- Promotion stacking must be explicit and deterministic.
- Every applied promotion is stored against the order.
- Merchant must be able to disable a promotion without deleting historical records.

Games

- Game rewards are funded/approved by merchant configuration.
- Games must have rate limits and anti-abuse controls.
- Game currency is separate from loyalty value.

Pricing

- Product prices are outlet-scoped and versioned.
- Historical orders retain historical price snapshots.

Permissions

- Financially sensitive actions require granular permission.
- All refunds, price overrides, deletions/voids and sensitive configuration changes are audited.

Data

- Tenant isolation is mandatory.
- Merchant export is required.
- Deletion/retention behavior must comply with contractual/legal requirements.

Launch plan

Starter: INR 499/month/outlet.

Includes core operating system and core customer-growth capabilities required to run a small cafe.

Future plans

Growth: approximately INR 999/month/outlet.

Pro: approximately INR 1,999/month/outlet.

These are commercial hypotheses and should be stored as plan configuration, not application constants.

Optional revenue streams

- premium messaging
- payment services where merchant opts in
- hardware sales
- premium integrations
- advanced analytics
- multi-outlet features

Subscription states

TRIAL -> ACTIVE -> PAST_DUE -> GRACE -> SUSPENDED -> CANCELLED

The exact grace period is a commercial decision. Historical invoices and tenant data must remain recoverable during suspended states.

Integration architecture

All third-party systems must use adapter interfaces. Core domain objects must not depend directly on provider-specific code.

Target classes

- Payment providers
- Swiggy
- Zomato
- WhatsApp/SMS
- Email
- Accounting systems
- Thermal printers
- Kitchen displays
- Existing POS migration/import sources

Adapter contract example

Each external adapter should define:

- authentication
- webhook/event handling
- retry policy
- rate limits
- mapping
- error classification
- health status
- reconciliation support

Swiggy/Zomato

Target capabilities:

82. Receive orders.
83. Map to canonical order model.

84. Push order status where supported.
85. Menu synchronization where supported.
86. Availability synchronization where supported.
87. Reconciliation.

Official API/partner availability must be validated during implementation. Do not rely on unofficial automation.

Payment provider abstraction

Support merchant-selected provider where technically available. Keep payment intent, payment transaction and settlement objects provider-neutral.

Messaging

Provider abstraction for WhatsApp/SMS/email/push. Campaign engine should not know the vendor implementation.

Objective

A cafe must continue core operations during internet outages.

Requirements

- Local persistent storage on POS client.
- Local transaction queue.
- Unique transaction IDs generated offline.
- Idempotent server writes.
- Sync status per record.
- Retry with backoff.
- Conflict detection.
- Operator-visible sync health.
- Local printer access.

Sync model

Prefer event or change-log based synchronization for transactional entities.

Example:

LOCAL ORDER_CREATED -> LOCAL PAYMENT_RECORDED -> LOCAL ORDER_COMPLETED

When online:

UPLOAD events -> SERVER ACK -> MATERIALIZE authoritative server state -> mark local events synced.

Conflict rules

- Completed financial transactions are append-only after close except controlled reversal/refund events.
- Configuration changes use version checks.
- Inventory uses transaction ledger rather than mutable absolute quantities alone.
- Customer profile merges require explicit conflict handling.

Failure cases to test

- Offline for 10 minutes.
- Offline for an entire shift.
- Device crash mid-order.
- Two terminals offline on same outlet.
- Internet returns during payment.
- Duplicate webhook after reconnection.
- Printer offline while POS remains available.

Core controls

- Tenant isolation at authorization and data-access layers.
- RBAC with granular permissions.
- Secure session management.
- MFA option/recommendation for owners.
- Sensitive-action reauthentication.
- Audit logs.
- Encryption in transit and at rest where appropriate.
- Secrets manager; never commit secrets.
- Backups and restore tests.
- Rate limiting.
- Abuse/fraud controls for rewards and games.

Audit events

At minimum:

- login/logout
- role/permission change
- price change
- tax configuration change
- refund
- void/cancel
- discount override
- loyalty adjustment
- gift card issue/redeem
- campaign change
- payment settings change
- data export
- support impersonation

Data minimization

Only collect customer fields that serve an identifiable product purpose. Do not require OTP or account creation merely because it is convenient for the platform.

Customer trust

Merchant-scoped data and export rights must be clearly defined in contracts and product terms.

Merchant KPIs

North-star:

- incremental gross profit influenced by platform.

Core:

- gross sales
- net sales
- order count
- AOV
- repeat purchase rate
- direct order share
- gross margin estimate
- discount rate
- refund rate
- cancellation rate
- food cost
- wastage
- customer LTV
- campaign ROI
- loyalty liability
- aggregator payout variance

Product KPIs

- activation rate
- time to first order
- time to go-live
- weekly active merchants
- daily active POS users
- order sync success rate
- offline transaction success rate
- sync conflict rate
- printer success rate
- crash/error rate
- support contacts per merchant

Growth KPIs

- leads contacted
- qualified leads
- demos
- trials
- paid conversions
- CAC
- payback period

- logo churn
- revenue churn
- net revenue retention

Dashboard principle

Every important dashboard should answer: What happened? Why? What should I do next?

Goal

Make replacing legacy restaurant software low-risk.

Migration modes

88. Fresh setup.
89. Import and parallel run.
90. Full cutover.

Data import categories

- business profile
- outlets
- categories
- products
- prices
- modifiers
- tax configuration
- recipes
- tables
- customers where export is available
- staff
- opening inventory where supported

Migration wizard

91. Connect/upload source.
92. Detect entities.
93. Map fields.
94. Show exceptions.
95. Validate.
96. Preview.
97. Import test dataset.
98. Merchant confirms.
99. Final import.
100. Parallel validation.
101. Cutover.

Success criteria

- No duplicate products caused by migration.
- Historical price snapshots remain correct.

- Test order works before cutover.
- Critical data exception count is zero or explicitly accepted.

Objective

Prove that a 30-50 order/day cafe can operate on Restaurant OS and that the product can improve repeat ordering without unacceptable operational friction.

P0 launch scope

Platform

- multi-tenancy
- tenant/outlet settings
- subscription
- authentication
- RBAC
- audit

Merchant

- onboarding
- dashboard
- menu
- POS
- orders
- billing
- payments recording
- tables
- KOT
- basic KDS
- inventory basics
- recipes basics
- staff
- printer support
- customer CRM
- loyalty
- coupons
- customer website
- QR ordering

Customer

- guest browsing
- guest checkout
- mobile identity
- order tracking
- rewards
- reorder

Explicitly not required to prove the MVP

- full Swiggy/Zomato menu sync
- advanced procurement
- complex membership
- consumer network
- AI assistant
- 20+ game catalog

One simple game may be used for pilot validation, but the game engine should be designed as a reusable domain.

Phase 0 - Foundation

- requirements
- architecture
- design system
- tenant model
- auth/RBAC
- domain model
- CI/CD
- observability

Phase 1 - Core operations

- menu
- POS
- order engine
- billing
- payments
- tables
- KOT/KDS
- offline sync
- printer integration

Phase 2 - Customer growth

- storefront
- QR
- CRM
- loyalty ledger
- coupons
- reorder
- referral

Phase 3 - Integration + retention

- Swiggy
- Zomato
- advanced inventory

- reconciliation
- campaigns
- games
- memberships
- gift cards

Phase 4 - Scale

- multi-outlet
- procurement
- advanced analytics
- accounting integrations
- hardware management

Phase 5 - Intelligence

- forecasting
- churn prediction
- dynamic merchandising
- AI assistant
- advanced automation

The following are intentionally not fully specified and require product/engineering sign-off before implementation if they materially affect architecture:

102. Final stack: framework, database, hosting, queue/event platform.
103. Exact POS local client approach: PWA, desktop shell, native Android, or hybrid.
104. Local multi-terminal synchronization protocol.
105. Exact printer protocols and supported models.
106. Payment providers at launch.
107. Exact Swiggy/Zomato API/partner capabilities.
108. GST/tax compliance implementation and accountant review.
109. Trial length and dunning/grace period.
110. Exact Growth/Pro plan entitlements.
111. Exact game catalog and game-economy rules.
112. Messaging vendors and regional cost assumptions.
113. Data retention periods.
114. Cloud region and disaster-recovery targets.
115. Customer consent language and notification preferences.
116. Merchant terms regarding customer data ownership/export.

Rule: unresolved product questions must become GitHub issues or ADRs. Do not silently invent business behavior.

Architecture Appendix

API Requirements

The API should be versioned and contract-driven.

Representative resources

GET/POST /v1/outlets

GET/POST /v1/menu/products

GET/POST /v1/orders

GET/PATCH /v1/orders/{id}

POST /v1/orders/{id}/payments

POST /v1/orders/{id}/refunds

GET/POST /v1/customers

GET /v1/customers/{id}

POST /v1/loyalty/earn

POST /v1/loyalty/redeem

GET/POST /v1/promotions

GET/POST /v1/campaigns

POST /v1/games/{id}/sessions

GET/POST /v1/tables

GET/POST /v1/inventory

GET/POST /v1/purchases

GET /v1/analytics/*

GET/POST /v1/integrations/*

Requirements

- Authentication required except public storefront endpoints.
- Tenant and outlet context enforced server-side.
- Idempotency for mutation endpoints that can be retried.
- Structured error responses.
- Pagination and filtering.
- Request tracing ID.
- Rate limits.

- Backward-compatible versioning policy.
- Webhook signature verification for external providers.

Public storefront

Public catalog and storefront APIs must expose only published data and never trust client-provided tenant/outlet authorization.

Database / Domain Model

Core entities

Tenant, Brand, Outlet, User, Role, Permission, StaffProfile, Device, Menu, Category, Product, ProductPrice, ModifierGroup, Modifier, Recipe, RecipeItem, Ingredient, Supplier, PurchaseOrder, PurchaseReceipt, InventoryLedger, InventoryBalance, Table, Order, OrderItem, Payment, Refund, Invoice, KitchenTicket, Customer, CustomerIdentity, LoyaltyAccount, LoyaltyLedger, Reward, Coupon, Promotion, Referral, Game, GameSession, GameReward, Membership, GiftCard, Campaign, CampaignAudience, CampaignEvent, Notification, Review, Subscription, IntegrationConnection, IntegrationEvent, AuditLog.

Key relationships

Tenant 1:N Outlet.

Tenant 1:N User.

Outlet 1:N Device.

Outlet 1:N Table.

Outlet 1:N Order.

Order 1:N OrderItem.

Order 1:N Payment.

Order 0:N Invoice.

Product N:N Modifier.

Product 0:1 Recipe.

Recipe 1:N RecipeItem.

Ingredient 1:N InventoryLedger.

Customer 1:1 LoyaltyAccount per outlet/brand scope as designed.

Customer 1:N Order.

Customer 1:N LoyaltyLedger.

Campaign 1:N CampaignEvent.

Game 1:N GameSession.

Critical implementation rules

- Use immutable transaction/ledger records for money, loyalty and inventory movements.
- Snapshot product name/price/tax configuration into order items so historical orders do not change when the catalog changes.
- Use idempotency keys for payments and order writes.
- Store source/provider references for external orders.
- Store timestamps in UTC with outlet timezone for rendering/reporting.
- Soft-delete configuration records where historical references must remain intact.

Event Model

Domain events

- TenantCreated
- OutletCreated
- ProductCreated
- ProductPriceChanged
- OrderCreated
- OrderPlaced
- OrderAccepted
- OrderPreparing
- OrderReady
- OrderCompleted
- OrderCancelled
- PaymentRecorded
- PaymentFailed
- RefundCreated
- InvoiceIssued
- KOTCreated
- KitchenTicketCompleted
- InventoryAdjusted
- StockConsumed
- CustomerCreated
- CustomerIdentified
- LoyaltyEarned
- LoyaltyRedeemed
- LoyaltyReversed
- CouponApplied
- ReferralAttributed
- GamePlayed
- GameRewardGranted

- CampaignScheduled
- CampaignDelivered
- CampaignRedeemed
- SubscriptionActivated
- SubscriptionPastDue

Event rules

- Events have globally unique IDs.
- Events are immutable.
- Consumers must be idempotent.
- Event version is explicit.
- Sensitive events must be auditable.

Example

OrderCompleted emits:

- RevenueRecognized
- InventoryConsumptionRequested
- LoyaltyEarnRequested
- CustomerOrderUpdated
- AnalyticsOrderUpdated
- CampaignEligibilityUpdated

The event bus is an internal contract; asynchronous side effects should not block order completion unless operationally required.

Integration Engineering Rules

Adapter pattern

Each provider is an adapter implementing a shared domain-facing interface.

Example interface concepts:

- receiveOrder
- updateOrderStatus
- syncMenu
- syncAvailability
- createPaymentIntent
- verifyWebhook
- fetchSettlement

Core services consume provider-neutral objects.

Retry policy

- Exponential backoff.

- Dead-letter queue after repeated failures.
- Alert merchant/platform admin when action remains unresolved.

Reconciliation

Every external money/order provider needs:

- provider order ID
- provider payment ID
- provider settlement ID where available
- timestamps
- gross value
- fees
- adjustments
- net value
- reconciliation status

Never assume external settlement equals internal order total.

Authorization Architecture

Use capability-based permissions with scopes.

Example capability:

```
orders.refund
```

Scope:

```
outlet:123
```

Do not encode authorization only as role names in backend code.

Recommended permission families:

- orders.*
- payments.*
- billing.*
- inventory.*
- menu.*
- kitchen.*
- customers.*
- loyalty.*
- promotions.*
- campaigns.*
- staff.*
- analytics.*
- integrations.*
- subscriptions.*

- settings.*

Audit every privileged action.

System Architecture

High-level

Client applications:

- Merchant web/POS
- Customer web/PWA
- Platform admin
- Optional local device/print agent

Backend domains:

- Identity/Auth
- Tenant
- Catalog
- Orders
- Payments
- Billing
- Kitchen
- Inventory
- Customers/CRM
- Loyalty
- Promotions
- Games
- Campaigns
- Analytics
- Integrations
- Notifications
- Subscriptions
- Support

Infrastructure concepts:

- Relational transactional database
- Cache
- Job queue/event bus
- Object storage
- Analytics warehouse/read model
- Observability stack

Principle

Keep the core domain modular. Third-party integration code stays behind adapters. Cross-module communication should prefer domain events over deep synchronous coupling.

Data separation

Tenant ID is mandatory on tenant-owned entities. Outlet ID is mandatory on outlet-scoped operational entities.

Recommended logical tiers

117. Presentation.
118. Application/use-case layer.
119. Domain layer.
120. Persistence/integration layer.
121. Async event processing.

The exact programming stack remains open and should be chosen by the engineering lead after evaluating offline requirements, hardware access and team competence.

Additional Product and QA Appendix

Screen-by-Screen Specifications

Merchant: Home

Purpose: Immediate operational and financial orientation.

Required blocks:

- Today sales.
- Orders.
- Average order value.
- Repeat purchase rate.
- Direct-order share.
- Alerts.
- Revenue opportunities.

Actions:

- Open POS.
- View live orders.
- Resolve alerts.
- Open inventory issue.
- View customer recovery opportunity.

Merchant: POS

Layout:

- left/center product/category navigation
- center cart/order ticket
- right payment/customer/table panel

Required actions:

- search product
- category filter
- modifiers
- quantity
- notes
- customer
- table
- discount
- split/merge
- hold
- payment
- print
- complete

Peak-hours rule: no nonessential modal should block transaction completion.

Merchant: Orders

Views:

- all
- live
- completed
- cancelled
- refunds
- channel

Filters:

- date
- outlet
- order type
- payment status
- source
- operator

Merchant: KDS

Columns:

- new
- preparing
- ready

Ticket contents:

- order number
- elapsed time
- priority
- items
- modifiers
- customer/table context
- source

Merchant: Menu

Capabilities:

- category tree
- product editor
- modifier groups
- prices
- availability
- images
- recipe link
- external channel mapping

Merchant: Inventory

Views:

- stock overview
- low stock
- wastage
- movements
- counts
- recipe consumption

Merchant: Customers

Top section:

- search by phone/name
- customer summary
- order history
- loyalty
- offers
- notes
- consent/preferences

Merchant: Customer detail

Must answer:

- who is this customer?
- how valuable are they?
- what do they buy?
- when did they last visit?

- what can we do to bring them back?

Merchant: Loyalty

- rules editor
- liabilities
- earn/redeem history
- reward catalog
- expiry settings
- economics warning

Merchant: Games

- game catalog
- enabled/disabled
- rules
- reward budget
- attempts
- eligibility
- performance

Merchant: Campaigns

Flow:

audience -> offer -> channel -> schedule -> approval -> preview -> send -> results.

Merchant: Analytics

Required top-level views:

- sales
- customers
- products
- loyalty
- campaigns
- margins
- channel mix
- reconciliation

Customer: Home

- cafe identity
- order CTA
- categories
- offers
- recommendations
- rewards balance
- game CTA
- reorder

Customer: Menu

- category navigation
- search
- product cards
- dietary/allergen information where merchant supplies it
- recommendation slots

Customer: Product

- image
- name
- description
- price
- modifiers
- add-ons
- recommendation
- add to cart

Customer: Checkout

- order summary
- customer mobile number optional
- payment method
- applied rewards/coupon
- terms/consent where required
- place order

Customer: Order status

- received
- accepted
- preparing
- ready
- completed
- issue/support CTA

Customer: Rewards

- balance
- how to earn
- available rewards
- redemption rules
- ledger/history

Customer: Games

- available games
- attempts remaining
- game result
- reward result
- next eligible play

Customer: Referral

- personal referral code/link
- reward terms
- pending referrals
- completed referrals

QA and Test Plan

Test layers

122. Unit tests - domain rules and calculations.
123. Integration tests - database, queue, payment adapters, printers where feasible.
124. Contract tests - API and integration adapters.
125. End-to-end tests - critical customer and merchant journeys.
126. Offline tests - POS and synchronization.
127. Security tests - authorization and tenant isolation.
128. Load tests - order ingestion and peak-hour flows.

P0 regression journeys

- merchant onboarding
- menu creation
- POS sale
- split payment
- discount
- invoice
- KOT
- KDS complete
- inventory consumption
- customer guest checkout
- customer identification
- loyalty earn
- loyalty redeem
- refund and loyalty reversal
- offline order
- reconnect sync
- printer failure/retry
- subscription activation

Tenant isolation tests

- merchant A cannot access merchant B data.
- outlet user cannot access unauthorized outlet.
- public storefront cannot read private merchant fields.
- support impersonation is audited.

Money correctness

- order total is deterministic.
- discount calculations are reproducible.
- taxes use stored configuration/version.
- split payments sum to order amount within configured rounding policy.
- refunds never exceed eligible captured amount.
- duplicate payment webhook is idempotent.

Loyalty correctness

- no duplicate reward event on retried order event.
- refund reverses configured earned points.
- redemption cannot produce negative balance unless explicitly supported.
- expired reward cannot redeem.

Offline correctness

Run tests for:

- one device offline
- two devices offline
- crash before sync
- crash after server accepts but before local ack
- duplicate sync attempt
- long outage
- clock skew
- printer unavailable

Performance targets

Exact SLOs should be set after stack selection. Target low-latency interaction for POS actions and mobile storefront browsing under typical Indian cafe connectivity.

Release gates

No production release if:

- critical payment defects exist.
- duplicate order creation exists.
- tenant isolation test fails.
- offline transaction loss exists.
- critical printer workflow fails.

Additional Architecture Appendix

Sequence Examples

Online POS order

129. Cashier creates order locally and sends create request.
130. API validates tenant/outlet/permissions.
131. Order service persists order and emits OrderCreated/OrderPlaced.
132. Payment service records payment.
133. KOT service creates kitchen ticket.
134. Inventory service reserves/consumes configured stock.
135. Loyalty service creates earn entry after qualifying completion.
136. Analytics consumes order event asynchronously.

Customer web order

137. Customer opens tenant/outlet storefront.
138. Catalog API returns published menu.
139. Customer creates cart client-side.
140. Checkout validates price/availability/promotions server-side.
141. Payment is initiated or merchant payment instructions are shown.
142. Order is created with idempotency key.
143. Kitchen workflow begins.
144. Customer receives order status updates.

Offline order

145. POS client detects offline state.
146. Transaction is committed to local durable store.
147. Local order ID is generated.
148. Local KOT/receipt output is attempted.
149. Transaction is appended to sync queue.
150. Connection returns.
151. Sync worker uploads transaction/event with idempotency key.
152. Server returns authoritative record.
153. Local store marks transaction synchronized.

Refund

154. Authorized operator selects order.
155. Server checks refund eligibility.
156. Refund record created.
157. External payment refund requested if applicable.
158. Inventory/loyalty/promotion adjustments produced according to rules.
159. Audit event recorded.
160. Customer notification queued.

Data Dictionary - Core Objects

Tenant

Business-level container.

Required: id, name, status, timezone, currency, created_at.

Outlet

Physical operating location.

Required: id, tenant_id, name, address, timezone, tax profile, status.

User

Login identity.

Required: id, tenant_id, auth identity, status.

StaffProfile

Operational employee information.

Required: user_id, outlet scope, role assignments, status.

Product

Sellable menu item.

Required: id, outlet/tenant scope, category, name, current published state.

ProductPrice

Versioned price record.

Required: product_id, effective_from, amount, tax mode, status.

Recipe

Ingredient consumption definition.

Required: product_id, version, yield assumptions.

Ingredient

Trackable raw material.

Required: unit, purchase unit, conversion, cost basis.

InventoryLedger

Immutable stock movement.

Examples: purchase, sale consumption, waste, adjustment, transfer, return.

Order

Canonical transaction intent and result.

Required: id, tenant_id, outlet_id, source, type, status, totals, operator/device context, created_at.

OrderItem

Snapshot of product at sale time.

Required: product snapshot, quantity, unit price, tax/discount allocation.

Payment

Payment attempt/record.

Required: order_id, method, status, amount, provider reference if any, idempotency key.

Invoice

Commercial document linked to order.

Required: number, issue timestamp, tax/business snapshot, totals.

Customer

Merchant-scoped customer profile.

Required: tenant/outlet scope, mobile identity, optional profile fields.

LoyaltyLedger

Immutable earn/redeem/reversal history.

Required: customer, event type, delta, source reference, balance_after, timestamp.

Promotion

Reusable discount/benefit rule.

Required: eligibility, calculation, stacking policy, validity.

GameSession

One play instance.

Required: game_id, customer/device context, result, reward, timestamps.

AuditLog

Security/business trace.

Required: actor, action, target, old/new values where appropriate, timestamp, request ID.

Developer Launch Checklist

- Confirm selected stack, environments and hosting.
- Confirm tenant/outlet authorization is tested.
- Confirm order state machine and idempotency tests.
- Confirm offline order persistence and recovery tests.

- Confirm printer/device validation matrix.
- Confirm payment/reconciliation behavior.
- Confirm loyalty/refund reversal behavior.
- Confirm game anti-abuse controls.
- Confirm audit logs and support tooling.
- Confirm backups and restore test.

Definition of Ready

- Feature has a PRD section.
- User role is known.
- Acceptance criteria exist.
- Data ownership and permissions are known.
- Offline impact is assessed.
- Analytics/audit events are identified.
- External dependencies are identified.

Definition of Done

- Acceptance criteria pass.
- Unit/integration tests exist where appropriate.
- Authorization is enforced server-side.
- Tenant/outlet boundaries are tested.
- Retry/idempotency behavior is defined.
- Relevant audit and analytics events exist.
- Documentation is updated.
- No critical unresolved defect remains.